Talofa

IN PROGRESS

A Samoan language learning app built on conversation.

Project brief by Vel'Shaun Byers

PLANNED STACK React, FastAPI, Python, speech to text, fine tuned open model, OAuth 2.0, pytest, GitHub Actions, Vercel, Railway

1. Why I Am Building This

My fiancée is Samoan, and there are no language applications that will help me learn how to speak her language. So I am dedicating my free time to building one.

I want to be able to speak to her in her language, and I want to be able to speak to our kids in that language.

I think conversations are the best way of learning. That is how children learn. The ability to infer what something may mean is the most comprehensive way of learning a language, so that is what I am building in.

2. What I Hope to Achieve

Talofa means hello in Samoan. It is the name of the app and the first word you see when it loads.

- **Current.** This is the core feature. I talk in English, and Samoan words flow into the conversation. I use context clues to work out what each new word means.
- **The strength of the current.** A slider controls how many Samoan words are mixed in, from a few to most of the sentence.
- **Word bank.** Words I work out go into my personal word bank, so I can see what I know and keep practicing it.

Success is being able to hold a real conversation with my fiancée and her family. This one is going to take a while, and I am fine with that.

3. Integrations

The real work on this app is the data. Samoan is a low resource language, so I have to build my own database of English and Samoan sentence pairs. This is called a parallel corpus.

- **Everyday conversations.** I record normal conversations with my fiancée in English. She, her close friends, and her family translate exactly what we said into Samoan.
- **Samoan films.** A speech to text model produces a rough Samoan transcript. Her family corrects it, which is less work than writing it from scratch, and then translates it into English. I will only use films that are in the public domain or that I have permission to use.
- **Speech to text.** I am still evaluating which open models handle Samoan well enough. Many popular models do not support it at all, which is why every transcript is checked by a native speaker.

It will take time to cover enough scenarios and subjects. Each one adds connections between the two languages.

4. AI Integrations

The plan is to end up with my own model. I am going to fine tune an open model on my corpus instead of training one from scratch, because I will have thousands of sentence pairs and not millions.

One candidate is NLLB 200, an open translation model that already includes Samoan [1]. A small open language model is the other option.

The model powers Current. It decides which words to swap into the conversation based on what is already in my word bank.

5. CI/CD Pipeline

Every push and pull request runs through GitHub Actions [2]. The workflow installs dependencies, runs the test suite, and builds the front end.

The React front end deploys to Vercel and the FastAPI back end deploys to Railway. A change only reaches the live site after the checks pass.

6. Testing

Back end tests are written with pytest [3] and run in the pipeline on every push.

The first tests cover the word substitution logic and the word bank, plus checks on the corpus itself, such as making sure every English line has a matching Samoan line.

7. Security

- Sign in uses OAuth 2.0 [4], so the app never stores a password.
- I use the OWASP Top Ten [5] as my checklist while building and reviewing.
- The recordings are private conversations. Raw audio never goes into the repository or the app. Only the translated sentence pairs that everyone involved agreed to include become part of the corpus.
- API keys and secrets live in environment variables on the host. They are never committed to the repository.

8. Accessibility

I build to the federal standard, Section 508. The Revised 508 Standards incorporate WCAG 2.0 Level AA by reference, so that is the bar I test against [6] [7].

- Every feature works with a keyboard alone, with a visible focus indicator.
- Text and controls meet the Level AA contrast ratios.
- Form fields have labels, and images and icons have text alternatives.
- Information is never carried by color alone.

9. Versioning and Lifecycle

I follow Semantic Versioning [8]. A bug fix bumps the patch number, a new feature bumps the minor number, and a breaking change bumps the major number.

Every release is tagged in GitHub with a short changelog entry. I am the only developer, but the rule does not change. It just means I am the one who enforces it.

The corpus is versioned as well, so I always know which data a model was fine tuned on. Samoan comes first. Current is built so the same engine could carry other languages later.

> **Current status.** In progress. Concept and data collection stage. The corpus has to exist before the model can.

References

[1] Meta AI Research. FLORES 200 and NLLB language list. https://github.com/facebookresearch/flores/blob/main/flores200/README.md

[2] GitHub. GitHub Actions documentation. https://docs.github.com/en/actions

[3] pytest documentation. https://docs.pytest.org/

[4] Internet Engineering Task Force. RFC 6749, The OAuth 2.0 Authorization Framework. https://datatracker.ietf.org/doc/html/rfc6749

[5] OWASP Foundation. OWASP Top Ten. https://owasp.org/www-project-top-ten/

[6] Section508.gov. Applicability and Conformance Requirements, Revised 508 Standards. https://www.section508.gov/develop/applicability-conformance/

[7] World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.0. https://www.w3.org/TR/WCAG20/

[8] Semantic Versioning 2.0.0. https://semver.org/